IT-5006 Fundamentals of Data Analytics

Team 8 Phase Two Report

Name	Student ID
Zeng Kaiwen	E1538151
Lim Feng Long	E1553038
Xu Hengchao	E1553234
Li Jin	E1582824
Li Qinkai	E1553577

National University of Singapore

AY 2025/2026

5th Mar 2026

Table of Contents

1. Problem Definition & Data Preparation

1.1 Problem Statement & Analytical Approach

Chicago has experienced crime challenges for a long time, thus making it important to understand the patterns of crime in order to come up with effective means of preventing them. With the availability of public crime data, historical records can be analyzed to better understand how crime incidents relate to factors such as time and location (Bogomolov et al., 2014).

The dataset is provided by the Chicago Police Department CLEAR system. This portal provides data records of crime that have occurred in Chicago from the year 2001 to the present. There are various attributes of the data, including details of the type of crime, date of occurrence, and location. Users can also filter the data based on specific years, among other attributes, from the website.

In our project, we define the task as a binary classification problem. The unit of analysis is Community Area + Week. In other words, instead of predicting at the individual crime-record level, we predict whether a specific community area will become a high-risk area in the following week. We define "high-risk" as the case where the total crime count of a community area in week t+1 is greater than or equal to the 75th percentile of weekly crime counts across all community areas in that same week. Overall, our analytical approach is to first aggregate the original crime records into weekly area-level observations, then build features based on crime intensity, temporal information, and recent historical trends, and finally use these features to predict whether the area will be high-risk in the next week.

1.2 Data Preprocessing & Cleaning Methodology

For phase two, we did not go back to the original Chicago crime data. Instead, we used the cleaned dataset chicago_crime_2015_2024_enriched.parquet, which had already been prepared earlier and was more suitable for modeling.

Before using it for training, we still did some checks on the dataset. We made sure the required columns were there and looked at whether key fields had missing values. In our case, Date and Community Area were especially important, so rows missing these fields were removed. We also converted Date into datetime format and made sure Community Area was stored in integer form, so that the records could be grouped properly later. After that, the records were arranged in time order and then aggregated into weekly observations. Since our task is to predict future crime risk, we kept the split chronological instead of random. Data from 2015 to 2024 was used for model development, while 2025 was reserved as the out-of-time holdout set for final evaluation. This setup is more realistic because the model learns from earlier years and is then evaluated on a later period.

1.3 Feature Engineering Strategy

Feature engineering was done after aggregating the data by Community Area + Week. This means the model does not directly use each individual crime record as one training sample. Instead, each sample represents the weekly crime situation of one area.

Several groups of features were created. We first focused on weekly crime statistics, including total_crime_count, theft_count, battery_count, and burglary_count. These variables give a direct picture of how active crime was in each area during that week, and also show what kinds of crimes were more common.

We also included behavior-related indicators such as arrest_rate and domestic_rate. These do not measure crime volume directly, but they still give some extra context about the incidents recorded in that area.

Third, we used time-related features, such as month and week_of_year, to capture seasonal or repeating temporal patterns. Finally, we created historical trend features, including lag_1, lag_2, rolling_mean_4, and rolling_mean_8. These features are meant to capture recent crime trends based on previous weeks only.

We also kept Community Area itself as a spatial feature, since different areas in Chicago have very different crime patterns. One important point is that these features were constructed only using information available up to the current week, while the label is based on the following week. This helps keep the prediction setup realistic and avoids using future information by mistake.

2. Model Implementation & Training

Four models were evaluated for crime risk prediction: Logistic Regression and Random Forest as interpretable baselines, and LightGBM and XGBoost as high-performance gradient boosting methods. All models were trained on an identical dataset and feature set for fair comparison.

The prediction task is framed as a binary classification problem at the Community Area × Week level: a community area is labeled high-risk if its crime count in week $t+1$ exceeds the 75th percentile across all community areas in that week. This relative hotspot formulation better reflects real-world law enforcement priorities than raw count prediction. The training set contains approximately 73.8% non-high-risk and 26.2% high-risk observations, with the validation set showing a nearly identical distribution — confirming that the chronological split introduced no significant sampling bias. Given the moderate class imbalance, hyperparameter tuning explored class-weighting strategies to guard against neglecting minority-class predictions.

2.1 Logistic Regression

Logistic Regression was implemented as the first baseline model due to its interpretability and computational efficiency. The preprocessing pipeline includes median imputation for missing values, standard scaling for numerical features, and one-hot encoding for the categorical feature representing community areas.

Hyperparameter tuning focused primarily on the regularization strength parameter C and optional class-weight adjustments. The best-performing configuration used a regularization parameter of C = 0.01 without additional class weighting. Despite its relatively simple linear structure, Logistic Regression achieved strong validation performance, with an accuracy of approximately 0.949, precision of 0.920, recall of 0.883, an F1-score of 0.901, and an ROC-AUC value of 0.986. These results demonstrate that the engineered temporal and spatial features provide substantial predictive power even for a relatively simple model.

2.2 Random Forest

To capture nonlinear relationships and complex feature interactions, a Random Forest classifier was implemented as the second baseline model. Random Forest is an ensemble learning method that constructs multiple decision trees and aggregates their predictions to improve generalization performance.

The preprocessing steps are similar to those used for Logistic Regression, including median imputation and one-hot encoding of categorical variables. Hyperparameter tuning explored several parameters such as the number of trees, maximum tree depth, minimum samples required for node splitting, and optional class weighting.

The best-performing Random Forest model used 200 trees with a maximum depth of 10 and a minimum split size of 10 samples. This configuration achieved slightly better performance than Logistic Regression, with an accuracy of approximately 0.951, precision of 0.940, recall of 0.868, an F1-score of 0.902, and an ROC-AUC of 0.987. Although the performance improvement is relatively small, the results highlight the ability of Random Forest to capture nonlinear relationships between historical crime patterns and future risk.

2.3 lightgbm

LightGBM is an improved algorithm based on Gradient Boosted Decision Trees (GBDT), employing a Leaf-wise growth strategy. Unlike traditional layer-wise growth, it prioritizes splitting leaf nodes that minimize loss most effectively. This enables the model to achieve higher accuracy and faster convergence when processing large-scale data.It utilizes GOSS (Gradient-based One-Side Sampling) and EFB (Exclusive Feature Binding).

Employ RandomizedSearchCV (random search) combined with 3-fold cross-validation: 1. num_leaves: Controls tree complexity, crucial for preventing overfitting. 2. learning_rate: Set between 0.01 and 0.1 to balance training speed and model stability. 3. min_child_samples: Prevents leaf nodes from forming on extremely small sample sets. Best parameter combination: {‘subsample’: 0.8, ‘num_leaves’: 20, ‘n_estimators’: 500, ‘min_child_samples’: 30, ‘max_depth’: 10, ‘learning_rate’: 0.01, ‘colsample_bytree’: 0.7}

2.4 XGBoost

XGBoost introduces several architectural innovations to the standard GBDT framework: Growth Strategy: Employs a Level-wise (layer-wise) growth strategy. It splits nodes layer by layer. While this approach incurs slightly higher computational costs, it produces a more balanced tree structure, effectively reducing overfitting risk. It utilizes not only first-order derivatives (residuals) but also second-order derivatives (Hessian) information, making the direction of gradient descent more precise. Incorporates L1 and L2 regularization into the objective function (controlling leaf node weights and counts), enhancing robustness in datasets with high feature noise. Includes built-in mechanisms for efficient handling of missing values and categorical features.

Strict adherence to chronological partitioning ensures the model “predicts the future,” preventing temporal leakage. Weight adjustment: Calculates the positive-to-negative sample ratio (ratio = (negative_count / positive_count)) and assigns it in to scale_pos_weight. This forces the model to impose greater penalties for misclassifying “high-risk (minority class)” samples. Accelerated computation: Enable tree_method=‘hist’. This histogram algorithm bins continuous features, significantly reducing training time while maintaining accuracy. Iteration control: Monitors the test set's AUC metric. Combined with EarlyStopping in callbacks, it automatically captures the peak of the model's generalization capability.

Hyperparameter Tuning: Similarly, RandomizedSearchCV was employed for multi-round sampling optimization. Optimal parameter combination: {‘subsample’: 0.8, ‘n_estimators’: 1000, ‘max_depth’: 2, ‘learning_rate’: 0.01, ‘gamma’: 0, ‘colsample_bytree’: 0.8}

3. Model Evaluation & Comparison

3.1 Standard Classification Metrics

We evaluated four candidate models: Logistic Regression, Random Forest, LightGBM, and XGBoost on the 2025 out-of-time holdout set using standard classification metrics. We calculated the Accuracy, Precision, Recall, F1-score, and ROC-AUC. Overall, all four models achieved strong predictive performance, with F1-scores clustered in a narrow range from 0.8616 to 0.8675 and ROC-AUC values above 0.983. In these four models, XGBoost achieved the highest F1-score (0.8675), while Random Forest achieved the highest ROC-AUC (0.9860). Logistic Regression also performed very strongly, with an F1-score of 0.8669 and a high ROC-AUC of 0.9844. LightGBM was slightly lower in F1-score (0.8616) but still remained competitive.

Logistic Regression and Random Forest produced very high precision values, which indicates fewer false positives when identifying high-risk weeks. In contrast, LightGBM and XGBoost achieved substantially higher recall, meaning they were better at capturing true high-risk weeks but at the cost of more false positives. Hence, Logistic Regression and Random Forest may be more suitable when false alarms are costly, whereas LightGBM and XGBoost may be preferable when missing high-risk areas is unacceptable.

3.2 Spatial Accuracy Evaluation

We computed Community-Area-level classification metrics and summarized the mean and standard deviation of F1-score across areas. The results indicate substantial spatial heterogeneity in model performance.

The mean area-level F1-scores were 0.2198 for Logistic Regression, 0.2118 for Random Forest, 0.2601 for LightGBM, and 0.2702 for XGBoost. At the same time, the standard deviation of area-level F1 was high for all four models. This shows that model performance was far from spatially uniform.

In the best-performing areas, some models achieved perfect F1-scores of 1.0, while in the worst-performing areas F1 dropped to 0, indicating that the model failed to identify any true high-risk weeks in those areas. Among the four models, XGBoost achieved the highest mean spatial F1-score, followed by LightGBM, suggesting that they offered slightly better spatial robustness. However, the large standard deviations confirm that spatial instability remained a major challenge for all models.

3.3 Temporal Accuracy Measures

To assess temporal accuracy, we evaluated monthly performance on the 2025 holdout set and summarized the mean and standard deviation of monthly F1-scores.We also summarized the mean monthly precision and recall. The monthly mean F1-scores were 0.8636 for Logistic Regression, 0.8611 for Random Forest, 0.8595 for LightGBM, and 0.8684 for XGBoost. Thus, XGBoost achieved the strongest performance, although the difference relative to the other models was small.

XGBoost had the lowest standard deviation of monthly F1-score (0.0189), followed by LightGBM (0.0233) and Random Forest (0.0245), while Logistic Regression had the highest monthly variation (0.0426). This indicates that the boosting-based models were more temporally stable. The monthly precision–recall profile is also consistent with the overall evaluation.

Hence, the boosting models were slightly more robust to month-to-month changes in crime patterns.

3.4 Model Robustness Analysis

We assessed model robustness through threshold sensitivity analysis by comparing performance at decision thresholds of 0.3, 0.5, and 0.7. For Logistic Regression and Random Forest, lowering the threshold from 0.5 to 0.3 increased recall and produced the highest F1-scores. Raising the threshold to 0.7 increased precision further but substantially reduced recall, which lowered F1-score for both models. This is a typical trade-off between precision and recall.

LightGBM behaved differently. Its F1-score increased as the threshold increased, rising from 0.8446 at 0.3 to 0.8616 at 0.5 and 0.8701 at 0.7. This suggests that LightGBM's probability outputs benefited from a higher decision threshold and that its classification behavior remained relatively stable across threshold changes. By contrast, XGBoost was the most threshold-sensitive model. At threshold 0.3, it achieved recall of 1 but precision of only 0.2629, resulting in a weak F1-score of 0.4164. At threshold 0.7, both precision and recall dropped to 0, producing an F1-score of 0. Its best performance occurred at the default threshold of 0.5, where F1-score reached 0.8675. Overall, LightGBM was the most stable under threshold variation, while XGBoost was the most sensitive to threshold selection.

3.5 Cross-validation

Because the prediction task is inherently time-dependent, we used a chronological validation strategy. To preserve the forecasting setting, we trained the models on the 2024 period and validated them on the 2025 period, thereby simulating a forward-looking deployment scenario.

Under this time-based validation setup, all four models achieved strong and consistent performance. Logistic Regression reached an accuracy of 0.9376, precision of 0.9465, recall of 0.8076, F1-score of 0.8715, and ROC-AUC of 0.9843. Random Forest achieved an accuracy of 0.9343, precision of 0.9684, recall of 0.7745, F1-score of 0.8607, and ROC-AUC of 0.9866. LightGBM achieved an accuracy of 0.9348, precision of 0.9178, recall of 0.8251, F1-score of 0.8690, and ROC-AUC of 0.9822. XGBoost produced the strongest overall validation results, with an accuracy of 0.9406, precision of 0.9336, recall of 0.8328, F1-score of 0.8803, and ROC-AUC of 0.9843.

These results show that the models generalized well from one year to the next under a realistic chronological split. XGBoost achieved the highest F1-score, indicating the best balance between precision and recall under time-based validation, while Random Forest achieved the highest ROC-AUC, suggesting the strongest ranking ability in distinguishing high-risk from non-high-risk weeks. LightGBM showed balanced performance with relatively strong recall. Overall, the chronological validation results are consistent with the later holdout evaluation and provide evidence that the models are able to transfer learned crime-risk patterns from past data to future.

4. Model Interpretation & Business Insights

4.1 Feature importance analysis

Feature importance analysis across our models (Random Forest & XGBoost) consistently identified three categories as most predictive, aligning with EDA findings:

Rank	Feature	Category	Key EDA Evidence
#1	Hour of Day	Temporal	10 PM–2 AM crime peak (heatmap, Sec. A)
#2	Community Area	Spatial	Top-10 areas = ~40% of all crimes (Sec. B/C)
#3	Day of	Temporal	Fri/Sat nights are significantly

	Week		elevated
#4	Police District	Spatial	Crime-type specialisation via Z-score (Sec. D)
#5	Location Description	Spatial	Arrest rates: 4%–70% across location types (Sec. E)
#6	Month / Season	Temporal	Summer 15–20% above winter baseline

4.2 Actionable Insights for Law Enforcement

- **Peak-Hour Surge Patrols:** Deploy additional resources Friday/Saturday 10 PM–2 AM — crime probability is 2.3× the weekly average during this window.
- **Geographic Prioritization:** Focus preventive resources on Austin, Near North Side, and Humboldt Park, which account for over 25% of total crime volume.
- **District-Specialized Training:** Districts 7, 10 & 11 show narcotics Z-scores >+2.5; align officer training to each district's crime-type profile (Sec. D).
- **Residential Arrest Improvement:** Arrest rates in residential locations fall below 5% despite high crime volume; improved evidence protocols and community liaison programs can close this gap (Sec. E).
- **Seasonal Staffing:** Pre-position additional officers before June each year to manage the consistent summer crime surge.

4.3 Limitations & Constraints

- **Reporting Bias:** No patrol density data means high-crime predictions may partly reflect over-policing rather than true prevalence — human review is essential before operational deployment.
- **Class Imbalance:** Rare crime types (arson, human trafficking) have insufficient samples; SMOTE and class-weight adjustments were applied but predictions for these categories remain less reliable.
- **Concept Drift:** COVID-19 caused a structural break in 2020 crime patterns. Models are trained on 2015–2022 data with a temporal test split (2023–2024) to limit leakage, but periodic retraining is required.
- **Ethical Constraints:** Predictive outputs must supplement — not replace — officer judgment. Feedback loops risk amplifying historical bias against already over-policed communities. Socioeconomic root causes are not captured in available data fields.

5. GitHub Link & Use of AI Tools

GitHub repository:

clear-train/Team8_IT5006_Predictive_Policing_AY2526Sem2

We used ChatGPT/Gemini to:

- Review grammar and improve the wording of a few sentences
- Modify the references to correct APA form
- Assist with code optimization, debugging, and technical documentation drafting
- The models implemented include Logistic Regression and Random Forest, and all

modeling decisions, feature engineering, and result analysis were completed by ourselves.

The ideas, analysis, and final content of the assignment are our own. The team is responsible for the content and quality of the submitted work.

6. References

1. Bogomolov, A., Lepri, B., Staiano, J., Oliver, N., Pianesi, F., & Pentland, A. (2014). Once upon a crime: Towards crime prediction from demographics and mobile data. *Proceedings of the 16th International Conference on Multimodal Interaction (ICMI '14)*, 427–434. https://doi.org/10.1145/2663204.2663254

2. Piquero, A. R., & Weisburd, D. (2010). Introduction. In A. R. Piquero & D. Weisburd (Eds.), *Handbook of quantitative criminology* (pp. 1–12). Springer. https://doi.org/10.1007/978-0-387-77650-7_1